

Stanse: May-analysis Framework for C Programs

Jan Obdržálek, Jiří Slabý and Marek Trtík

Faculty of Informatics, Masaryk University, Brno, Czech Republic
obdrzalek@fi.muni.cz, xslaby@fi.muni.cz, trtik@mail.muni.cz

Abstract. STANSE is a general may-analysis framework and software tool for verification of C programs. It is designed to (a) verify a large software projects, like Linux kernel, and (b) to provide quick implementation of new validation techniques with minimal effort. STANSE currently implements two validation techniques, one verifying problems specified by finite automata, and the other for detecting deadlocks. STANSE was tested on the recent Linux kernel, where it has found tens of previously undiscovered bugs. STANSE is freely available under GPL2 license.

1 Introduction

Bug-finding based on static analysis in the last decade have finally become of age. One of the first papers to really stir interest was [1]. There it was shown that, using known techniques, it is possible to effectively find many interesting bugs in real-world code, as demonstrated on Linux. This work eventually led to a successful commercial tool called COVERITY PREVENT [4]. Over the years, many other successful tools, like KLOCWORK [5], appeared.

STANSE can be thought of in two ways. Firstly, STANSE is a framework for implementing different may-analysis algorithms. It contains a C parser, common data structures like AST, CFG and call-graph, libraries for manipulating and traversing these structures. The tool contains front ends for (a) configuring the verification (specifying sources, checkers, etc.), and (b) error reporting mechanisms. Secondly, STANSE can be immediately used as a static analysis tool for detecting errors in locking discipline, pointer handling etc. This functionality comes from the built-in checkers.

2 Architecture

STANSE is a fully opened tool and framework for static validation model checking written predominantly in JAVA and distributed freely under GPL2 license. It was primarily designed to be easy used for (a) verification of a large projects, like Linux kernel, and (b) for a quick implementation of new validation techniques with minimal effort. How these design goals were achieved in the implementation of the STANSE is discussed in following subsections.

2.1 Concept of Checkers

In STANSE a checker is implementation of some concrete validation technique. There can be many checkers simultaneously running in STANSE, but they do not see each other. They proceed independently. The checkers share read-only lazy representation of checked program and they are gained an access to the algorithms providing common types of navigation in it (see 2.3). This is simply done by implementing JAVA interface for checkers.

The checkers are integrated to the framework of STANSE using concrete factory design pattern. So to insert a new checker to the framework of STANSE one needs to (a) implement general checker interface, and (b) register the checker to the checkers' factory of the framework. Then the checker has full access to the features of the STANSE and is immediately ready for checking projects like Linux kernel (when the validation technique itself is properly implemented by the checker, of course).

2.2 Independent Framework

The tool STANSE contains the framework STANSE, which is responsible for executing the program's analysis itself. The framework (a) accepts a configuration of the analysis, which references the sources of a program to be verified and list of checker to be applied, (b) executes the analysis itself by instantiating appropriate checkers and running them on the sources, and (c) collects error reports from the checkers, which are finally returned as the result of the analysis.

Actually all discussed steps of the framework are quite complex. For example configuration of the analysis can reference sources in different ways (Makefiles, Batchfiles, Directories, Single file, etc.) and checkers in the list are usually passed additional checker-specific data, and checkers are specified type of the analysis (intra/inter-procedural). Also the step (b) can be executed in different ways. The checkers may be executed in parallel or in sequence, and whole the analysis may be synchronous or asynchronous.

The tool STANSE actually is a concrete possible use of the STANSE framework. The tool thus only define (a) command line commands and GUI dialogs for specifying the configuration of the analysis, and (b) the formatted output of error reports back to the console and GUI dialogs for interactive tracing of the error reports. Since the framework is designed to be independent of some concrete use, the approach gives other developers possibility to reuse the STANSE framework for implementing another validation tools, or for example plug-ins for some IDEs.

2.3 Lazy Internals

STANSE framework provides common representations, like AST, CFG, and Call-graph, of verified program. The checkers can access these internal structures at any time during their life time. Those structures are called lazy, since they are instantiated as needed by checkers. For example, when call-graph or CFG of some function is not accessed by any of the checkers, then the graph or the function

is never instantiated in the memory. The data are actually created when first time accessed. The instantiated data can be then discarded only by streaming (see 2.4).

The framework of STANSE provides the checkers also library of algorithms providing common types of navigation in these structures. The checkers may traverse function in depth-first, breadth-first order, or they may traverse paths in functions in forward and backward direction. The framework further provides matching of actual to formal parameters in the call sites, and checkers may search for specific subtrees of AST by specifying the query in a simple language designed for such purpose (pattern matching technique).

2.4 Streaming

The programs used in practice (like Linux kernel) typically consists of hundreds or thousands of modules (compilation units). It is practically impossible for validation tools to store representations of all the modules in memory. Thus STANSE provides streaming of the internal structures (see 2.3) on module basis. It means, that when checker accesses some data related to some source module, then whole module is streamed in. Since internal structures are lazy 2.3, there is no time or memory bottleneck. The data of some module which are not necessary anymore, or which are not accessed by checkers as often as other data, then the data are completely discarded from the memory. Discarded are all the data related to the module. When discarded module is accessed again, the module is streamed in again, as before.

2.5 Languages

Although STANSE may contains many parsers of programming languages, currently the STANSE contains only C parser (implementing ISO/ANSI C99, including many GNU C extensions), so the framework can be used only for C programs now. But other languages may be added in the future.

toto by se zde snad ani
uvadet nemuselo

3 Running Stanse on the Linux Kernel

Since STANSE is intended to be a general validation framework for software used in nowadays practice, it must also be practically approved, that the intention is reached. We used STANSE on Linux kernel. There were three goals to be examined during the experimentation: (a) ensure, that framework STANSE is sufficiently fast and flexible for validation of large software, (b) verify, the framework is of the maximal use when developing new checkers, and (c) find some bugs in Linux kernel, of course.

For purpose of this experimentation, there must be at least one checker available. So we wrote two checkers AUTOMATONCHECKER, and THREADCHECKER, both with general use (e.i. not only for Linux kernel). These checkers are described in the following subsections. The results from the experimentation are discussed in section 4.

3.1 Automaton Checker

This checker is heavily influenced by [1]. It takes, as an input, a set of finite-state automata, which describe some property which we want to check. Properties like locking discipline, interrupt management, null pointer dereference, dangling pointers and many others can be handled this way.

Compared to the implementation described in [1] and [2], the technique used differs in several aspects. Here is a list of the most important differences: (1) we do not use metacompilation, (2) automata are not input-language specific, (3) the interprocedural analysis is done within a single compilation unit.

3.2 Thread Checker

THREADCHECKER aims to check for possible deadlocks in concurrent programs. The technique used is based on the notions of locksets of [3] and deadlock detection by looking for cycles in resource allocation graphs (RAG). THREADCHECKER first tries to identify the parts of the code which can run in parallel, as different threads. Then it builds a set of *dependence graphs* for each such thread. A dependence graph statically represents the possible locksets during one execution of a thread. Dependency graphs are then combined and all cycles examined, by building the corresponding RAGs.

4 Results

uvest parametry pocitace

The running time of STANSE on a common desktop machine was under 72 minutes (AUTOMATONCHECKER was even able to process multiple files per second), and the memory usage of the Java processes oscillated between 200 and 600 MiB. These profile statistics shows, that STANSE framework is fast and flexible, and it can be successfully applied to large projects like Linux kernel.

Implemented checkers AUTOMATONCHECKER and THREADCHECKER strongly uses functionality provided by STANSE framework. It can be seen, by inspecting their source code, that the checkers implement just the technique. The data structures and algorithms in the STANSE framework, supporting implementation of validation techniques (see 2.3), was sufficient for checkers' needs.

And finally, more than 50 bugs¹ were reported to kernel developers, most with the appropriate patches, and fixed in the new kernel releases. Some of these bugs remained undiscovered for seven years². This is despite the fact, that each compilable build of the Linux kernel is being checked with very successful commercial tool COVERITY PREVENT.

uvest jeste procentualni
zastoupeni false-positives

procenta false positives:

- locking_irq - $100 * (41 + 48 + 44) / ((41 + 48 + 44) + (26 + 23 + 6)) = 70.74468085106383\%$
- memory - $100 * (18 + 13 + 29) / ((18 + 13 + 29) + (17 + 7 + 23)) = 56.07476635514019\%$

¹ A list of bugs is available at <http://stanse.fi.muni.cz/bugs.html>

² <http://lkml.org/lkml/2009/3/11/380>

5 Acknowledgements

Jan Kučera is the author of the THREADCHECKER. We would like to thank Linux kernel developers who helped us sorting out false positives from real bugs, and Cyrill Gorcunov for STANSE alpha testing and useful suggestions which helped to make STANSE better. All authors are supported by the research centre Institute for Theoretical Computer Science (ITI), project No. 1M0545.

References

1. Dawson Engler, Benjamin Chelf, Andy Chou, and Seth Hallem. Checking system rules using system-specific, programmer-written compiler extensions. In *OSDI'00*, pages 1–16, 2000.
2. Seth Hallem, Benjamin Chelf, Yichen Xie, and Dawson Engler. A system and language for building system-specific, static analyses. In *PLDI'02*, pages 69–82. ACM, 2002.
3. Jan Wen Voun, Ranjit Jhala, and Sorin Lerner. RELAY: static race detection on millions of lines of code. In *ESEC-FSE'07*, pages 205–214. ACM, 2007.
4. COVERITY PREVENT. <http://www.coverity.com/products/>.
5. KLOCWORK. <http://www.klocwork.com/products/>.